```
"""
main.py — ВЕСЬ проект в одном файле: Telegram-бот (aiogram) + юзербот (Pyrogram).

Запуск:            python main.py
Получить сессию:   python main.py session     (один раз, на телефоне/ПК, НЕ на Re

Переменные окружения (Render -> Environment):
  BOT_TOKEN       токен от @BotFather
  API_ID          с https://my.telegram.org  (API development tools)
  API_HASH        оттуда же
  SESSION_STRING  строка сессии юзербота (см. "python main.py session")
  PYTHON_VERSION  3.11.9   (рекомендую, чтобы всё гарантированно собралось)

Команды юзербота (в любом чате, от твоего аккаунта):
  .spam [1-500] [текст]   .heart [текст]   .stop
  .slots   .dice   .darts
  .freeze [ник/id]   .snos [ник/id]   .unfreeze [Task ID]
  .mute [время]   .unmute
  .info [ник/id]
Цель команды: ник/id в команде -> reply на сообщение -> в личке собеседник.
"""
from __future__ import annotations

import asyncio
import logging
import os
import random
import re
import sys
import textwrap
import time
import uuid
from dataclasses import dataclass, field
from html import escape
from typing import Awaitable, Callable, Optional

# .env подхватываем, если он есть (на Render переменные задаются в панели — тоже
try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    pass

from aiogram import Bot, Dispatcher, F, Router
```

```
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode
from aiogram.exceptions import TelegramBadRequest, TelegramRetryAfter
from aiogram.filters import CommandStart
from aiogram.types import (
    CallbackQuery, ChosenInlineResult, InlineKeyboardButton, InlineKeyboardMarkup
    InlineQuery, InlineQueryResultArticle, InputTextMessageContent,
    Message as BotMessage,
)
from aiohttp import web
from pyrogram import Client, filters
from pyrogram.enums import ChatType, ParseMode as PM
from pyrogram.errors import FloodWait, MessageNotModified, RPCError
from pyrogram.handlers import MessageHandler
from pyrogram.types import Message, User

logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(name)s | %(leveln
log = logging.getLogger("app")

# ═══════════════════════════════════════════════════════════════════════
#                                НАСТРОЙКИ
# ═══════════════════════════════════════════════════════════════════════


def _int(name: str) -> int:
    try:
        return int(os.getenv(name, "0").strip() or 0)
    except ValueError:
        return 0


BOT_TOKEN: str = os.getenv("BOT_TOKEN", "").strip()
API_ID: int = _int("API_ID")
API_HASH: str = os.getenv("API_HASH", "").strip()
SESSION_STRING: str = os.getenv("SESSION_STRING", "").strip()

BOT_NAME = "Control Panel Userbot"
P = "."                                  # префикс команд
SPAM_MAX_COUNT = 500
SPAM_DELAY_RANGE = (1.5, 2.5)            # пауза между сообщениями спама (сек)
ANIM_DELAY_RANGE = (1.0, 2.0)            # пауза между кадрами анимаций (сек)
DEFAULT_MUTE_SECONDS = 600               # .mute без времени = 10 минут
HEART_FILL, HEART_BG = "❤️", "⬛"
FREEZE_MINUTES_RANGE = (30, 60)          # .freeze: «в течение N минут»
SNOS_HOURS_RANGE = (24, 72)              # .snos: «в течение N часов»

# ═══════════════════════════════════════════════════════════════════════
```

```
#                          ГЛОБАЛЬНОЕ СОСТОЯНИЕ
# ═══════════════════════════════════════════════════════════════════════════
OWNER_ID: int = 0                              # определяется автоматически по аккаунт
client: Optional[Client] = None                # сам юзербот
BOT_USERNAME: str = ""                         # username бота (нужен для inline-анима
TASKS: set = set()                             # отменяемые задачи (.stop)
_BG: set = set()                               # служебные задачи (удаление «вспышек»)
FREEZES: dict = {}                             # Task ID -> данные анимации freeze/sno
MUTED: dict = {}                               # (chat_id, user_id) -> unix-время конц
SCORE = {"total": 0}                           # очки слотов за сессию
REPORTS: dict = {}                             # report_id -> Report (ждут «📪 Опублик

ReportSender = Callable[[str, str], Awaitable[None]]
FreezeCanceller = Callable[[Optional[str]], Awaitable[int]]
_report_sender: Optional[ReportSender] = None
_freeze_canceller: Optional[FreezeCanceller] = None

# ═══════════════════════════════════════════════════════════════════════════
#                          ЮЗЕРБОТ: вспомогательные функции
# ═══════════════════════════════════════════════════════════════════════════


def _fw(e: FloodWait) -> int:
    """Секунды ожидания из FloodWait (атрибут называется по-разному в версиях)."""
    return int(getattr(e, "value", getattr(e, "x", 5)))


def spawn(coro) -> asyncio.Task:
    """Запускает отменяемую фоновую задачу и регистрирует её для .stop."""
    task = asyncio.get_running_loop().create_task(coro)
    TASKS.add(task)
    task.add_done_callback(TASKS.discard)
    return task


def _bg(coro) -> None:
    """Служебная задача, которую .stop НЕ отменяет."""
    task = asyncio.get_running_loop().create_task(coro)
    _BG.add(task)
    task.add_done_callback(_BG.discard)


async def safe_edit(msg: Message, text: str) -> None:
    """Редактирует сообщение, игнорируя «не изменилось» и FloodWait."""
    try:
        await msg.edit_text(text, parse_mode=PM.MARKDOWN)
    except MessageNotModified:
```

```
        pass
    except FloodWait as e:
        await asyncio.sleep(_fw(e))
    except RPCError as e:
        log.warning("edit failed: %s", e)


async def _delete_later(msg: Message, ttl: float) -> None:
    await asyncio.sleep(ttl)
    try:
        await msg.delete()
    except RPCError:
        pass


async def flash(chat_id, text: str, ttl: float = 4) -> None:
    """Короткое служебное сообщение, которое само удаляется через ttl секунд."""
    try:
        m = await client.send_message(chat_id, text, parse_mode=PM.MARKDOWN)
    except RPCError:
        return
    _bg(_delete_later(m, ttl))


async def drop(message: Message) -> None:
    """Удаляет сообщение с командой (чтобы чат оставался чистым)."""
    try:
        await message.delete()
    except RPCError:
        pass


def esc(text) -> str:
    """Экранирует символы Markdown, чтобы имена/био не ломали разметку."""
    return re.sub(r"([\\_*~|`\[\]()])", r"\\\1", str(text))


def full_name(user: User) -> str:
    return " ".join(p for p in (user.first_name, user.last_name) if p) or "Без им


def args_of(message: Message) -> list:
    """Аргументы команды без самой команды."""
    return (message.command or [])[1:]


async def resolve_user(message: Message, arg: Optional[str]) -> Optional[User]:
```

```
    """Ник / id / reply -> User."""
    try:
        if arg:
            arg = arg.strip()
            key = int(arg) if arg.lstrip("-").isdigit() else arg.lstrip("@")
            return await client.get_users(key)
        if message.reply_to_message and message.reply_to_message.from_user:
            return message.reply_to_message.from_user
    except (RPCError, KeyError, ValueError) as e:
        log.info("resolve_user: %s", e)
    return None


async def resolve_target(message: Message, arg: Optional[str] = None) -> Optional
    """Как resolve_user, но в личном чате без аргументов берёт собеседника (ник н
    user = await resolve_user(message, arg)
    if user and not user.is_self:
        return user
    if not arg and message.chat.type == ChatType.PRIVATE:
        try:
            u = await client.get_users(message.chat.id)
            return None if u.is_self else u
        except (RPCError, KeyError, ValueError):
            return None
    return None


def parse_human_duration(tokens: list) -> Optional[int]:
    """'10 min' / '10m' / '2h' / '30s' / '1d' / '15' (=минуты; русские единицы то
    text = "".join(tokens).lower().replace(" ", "")
    m = re.fullmatch(r"(\d+)([a-za-яё]*)", text)
    if not m or int(m.group(1)) <= 0:
        return None
    n, unit = int(m.group(1)), m.group(2)
    if not unit:
        return n * 60
    mult = {"с": 1, "s": 1, "м": 60, "m": 60, "ч": 3600, "h": 3600, "д": 86400, "
    return n * mult if mult else None


def fmt_duration(sec: int) -> str:
    h, rem = divmod(sec, 3600)
    m, s = divmod(rem, 60)
    parts = [f"{h}ч" if h else "", f"{m}м" if m else "", f"{s}с" if s and not h e
    return " ".join(p for p in parts if p) or "0с"
```

```
# ═══════════════════════════════════════════════════════════════════════
#                                .stop
# ═══════════════════════════════════════════════════════════════════════
async def cmd_stop(_, message: Message):
    """Мгновенно отменяет спам/анимации и активные freeze/snos."""
    count = len(TASKS)
    for t in list(TASKS):
        t.cancel()
    if _freeze_canceller:
        count += await _freeze_canceller(None)
    await drop(message)
    await flash(message.chat.id, f"⛔ **Остановлено задач:** `{count}`", 3)


# ═══════════════════════════════════════════════════════════════════════
#                                .spam
# ═══════════════════════════════════════════════════════════════════════
async def spam_job(chat_id: int, count: int, text: str):
    """Шлёт сообщения с рандомной паузой; при FloodWait ждёт и повторяет."""
    sent = 0
    while sent < count:
        try:
            await client.send_message(chat_id, text, parse_mode=PM.DISABLED)
            sent += 1
        except FloodWait as e:
            await asyncio.sleep(_fw(e) + 1)
            continue
        except RPCError as e:
            await flash("me", f"❌ Спам остановлен: `{esc(e)}`", 10)
            return
        if sent < count:
            await asyncio.sleep(random.uniform(*SPAM_DELAY_RANGE))


async def cmd_spam(_, message: Message):
    parts = (message.text or "").split(maxsplit=2)
    if len(parts) < 3 or not parts[1].isdigit() or not 1 <= int(parts[1]) <= SPAM
        await safe_edit(message, f"ℹ️ Использование: `{P}spam [1-{SPAM_MAX_COUNT}
        _bg(_delete_later(message, 5))
        return
    chat_id = message.chat.id
    await drop(message)
    spawn(spam_job(chat_id, int(parts[1]), parts[2]))


# ═══════════════════════════════════════════════════════════════════════
#                                .heart
```

```
# ═══════════════════════════════════════════════════════════════════════
HEART_MASK = [
    "01100110",
    "11111111",
    "11111111",
    "11111111",
    "01111110",
    "00111100",
    "00011000",
]


def build_heart(text: str) -> str:
    """Сердце из смайликов; текст (до 2 строк по 12 символов) — в центре."""
    lines = textwrap.wrap(text, width=12)[:2]
    text_rows = {}
    if len(lines) == 1:
        text_rows[3] = lines[0]
    elif len(lines) == 2:
        text_rows[2], text_rows[3] = lines
    out = []
    for i, mask in enumerate(HEART_MASK):
        if i in text_rows:
            out.append(f"{HEART_FILL} {text_rows[i]} {HEART_FILL}")
        else:
            out.append("".join(HEART_FILL if c == "1" else HEART_BG for c in mask
    return "\n".join(out)


async def cmd_heart(_, message: Message):
    text = " ".join(args_of(message)).strip()
    await drop(message)
    await client.send_message(message.chat.id, build_heart(text), parse_mode=PM.D


# ═══════════════════════════════════════════════════════════════════════
#                                .slots
# ═══════════════════════════════════════════════════════════════════════
SLOT_SYMBOLS = ["🍒", "🍋", "🍉", "🔔", "💎", "7️⃣"]
SLOT_VALUE = {"🍒": 10, "🍋": 15, "🍉": 20, "🔔": 30, "💎": 50, "7️⃣": 100}


def slot_frame(reels: list, footer: str = "🎲 Крутим...") -> str:
    return (
        "🎰 **КАЗИНО** 🎰\n"
        "╔═══════════╗\n"
        f"║ {reels[0]} | {reels[1]} | {reels[2]} ║\n"
```

```
        "╚═══════════╝\n"
        f"{footer}"
    )


def slot_score(reels: list) -> tuple:
    """Возвращает (очки, заголовок результата)."""
    a, b, c = reels
    if a == b == c:
        return SLOT_VALUE[a] * 5, "🏆 **ДЖЕКПОТ!**"
    for sym in set(reels):
        if reels.count(sym) == 2:
            return SLOT_VALUE[sym], "✨ **Пара!**"
    return 0, "💨 **Мимо.**"


async def slots_job(msg: Message):
    reels = [random.choice(SLOT_SYMBOLS) for _ in range(3)]
    final = [random.choice(SLOT_SYMBOLS) for _ in range(3)]
    for step in range(8):  # сначала крутятся все, затем барабаны встают по очере
        for i in range(3):
            reels[i] = final[i] if step >= 4 + i else random.choice(SLOT_SYMBOLS)
        await safe_edit(msg, slot_frame(reels))
        await asyncio.sleep(0.6)
    points, title = slot_score(final)
    SCORE["total"] += points
    await safe_edit(msg, slot_frame(final, f"{title}\n➕ Очки за раунд: **{points


async def cmd_slots(_, message: Message):
    await safe_edit(message, slot_frame(["❔"] * 3))
    spawn(slots_job(message))


# ═══════════════════════════════════════════════════════════════════
#                          .dice / .darts
# ═══════════════════════════════════════════════════════════════════
async def cmd_dice(_, message: Message):
    emoji = "🎯" if message.command[0].lower() == "darts" else "🎲"
    chat_id = message.chat.id
    await drop(message)
    await client.send_dice(chat_id, emoji)  # нативная анимация Telegram


# ═══════════════════════════════════════════════════════════════════
#                     .freeze / .snos / .unfreeze
#  Юзербот не умеет ставить инлайн-кнопки, поэтому запускает анимацию через
```

```
#  inline-режим бота (сообщение будет с пометкой «via @бот»). Саму анимацию
#  рисует бот — см. раздел «БОТ» ниже.
# ═══════════════════════════════════════════════════════════════════════
async def cmd_freeze(_, message: Message):
    mode = "snos" if message.command[0].lower() == "snos" else "freeze"
    arg = args_of(message)[0] if args_of(message) else None
    user = await resolve_target(message, arg)
    chat_id = message.chat.id
    if not user or not BOT_USERNAME:
        await safe_edit(message, f"ℹ️ `{P}{message.command[0]}` — ответом на сооб
        _bg(_delete_later(message, 5))
        return

    task_id = "FRZ-" + uuid.uuid4().hex[:8].upper()
    if mode == "freeze":
        minutes = random.randint(*FREEZE_MINUTES_RANGE)
        total, label = minutes * 60, f"{minutes} мин."
    else:
        hours = random.randint(*SNOS_HOURS_RANGE)
        total, label = hours * 3600, f"{hours} ч."

    FREEZES[task_id] = {
        "user": user,
        "mode": mode,
        "chat_id": chat_id,
        "total_seconds": total,    # на сколько «тикает» таймер
        "label": label,            # «45 мин.» / «48 ч.» — для текста
        "inline_id": None,         # заполнит бот, когда сообщение уйдёт в чат
        "task": None,              # задача анимации (создаст бот)
        "cancelled": False,
    }
    await drop(message)
    try:
        res = await client.get_inline_bot_results(BOT_USERNAME, f"fz {task_id}")
        await client.send_inline_bot_result(chat_id, res.query_id, res.results[0]
    except (RPCError, IndexError) as e:
        FREEZES.pop(task_id, None)
        await flash("me", f"❌ Не удалось отправить анимацию: `{esc(e)}`\nПроверь
        return
    _bg(_freeze_watchdog(task_id))


async def _freeze_watchdog(task_id: str):
    """Если бот не получил chosen_inline_result — подсказываем, что включить."""
    await asyncio.sleep(8)
    info = FREEZES.get(task_id)
    if info and info["inline_id"] is None and not info["cancelled"]:
```

```
        await flash("me", "⚠️ Анимация не стартовала. В @BotFather: /setinlinefee


async def cmd_unfreeze(_, message: Message):
    args = args_of(message)
    n = await _freeze_canceller(args[0].upper()) if (args and _freeze_canceller)
    await drop(message)
    await flash(message.chat.id, "✅ Операция отменена" if n else f"❌ Task ID не


# ═══════════════════════════════════════════════════════════════════════
#                              .mute / .unmute
# ═══════════════════════════════════════════════════════════════════════
async def _mute_target(message: Message):
    """
    `.mute 10 min` — в личке: собеседник; в группе: автор сообщения, на которое о
    `.mute @ник 2h` / `.mute 123456789 1d` — явный пользователь (необязательно).
    Возвращает (user, секунды | None если время задано неверно).
    """
    tokens = args_of(message)
    arg = None
    if tokens and (tokens[0].startswith("@") or (tokens[0].isdigit() and len(toke
        arg, tokens = tokens[0], tokens[1:]
    duration = parse_human_duration(tokens) if tokens else DEFAULT_MUTE_SECONDS
    return await resolve_target(message, arg), duration


async def cmd_mute(_, message: Message):
    user, duration = await _mute_target(message)
    chat_id = message.chat.id
    await drop(message)
    if not user or not duration:
        await flash(chat_id, f"ℹ️ `{P}mute 10 min` — в личке или ответом на сообщ
        return
    MUTED[(chat_id, user.id)] = time.time() + duration
    await flash(chat_id, f"🔇 **{esc(full_name(user))}** — автоудаление сообщений


async def cmd_unmute(_, message: Message):
    user, _d = await _mute_target(message)
    chat_id = message.chat.id
    await drop(message)
    if user and MUTED.pop((chat_id, user.id), None):
        await flash(chat_id, f"🔊 Мьют снят: **{esc(full_name(user))}**", 4)


async def _muted_check(_, __, m: Message) -> bool:
```

```
    if m.outgoing or not m.from_user:
        return False
    key = (m.chat.id, m.from_user.id)
    exp = MUTED.get(key)
    if exp is None:
        return False
    if exp < time.time():  # срок вышел — чистим
        MUTED.pop(key, None)
        return False
    return True


muted_filter = filters.create(_muted_check)


async def delete_muted(_, m: Message):
    """Удаляет входящее сообщение замьюченного (в группах нужны права на удаление
    try:
        await m.delete(revoke=True)
    except FloodWait as e:
        await asyncio.sleep(_fw(e))
    except RPCError:
        pass


# ═══════════════════════════════════════════════════════════════════════
#                   .info (только публичные данные профиля)
# ═══════════════════════════════════════════════════════════════════════
@dataclass
class Field:
    label: str
    value: str
    sensitive: bool = False  # True -> при публикации уйдёт под спойлер


@dataclass
class Report:
    target_chat_id: int
    sections: list = field(default_factory=list)  # [(заголовок, [Field, ...]), .
    sources: list = field(default_factory=list)


STATUS_RU = {
    "ONLINE": "в сети", "OFFLINE": "офлайн", "RECENTLY": "был(а) недавно",
    "LAST_WEEK": "на этой неделе", "LAST_MONTH": "в этом месяце", "LONG_AGO": "да
}
```

```
def _yn(v) -> str:
    return "да" if v else "нет"


async def collect_report(user: User, chat_id: int) -> Report:
    """Собирает ТОЛЬКО то, что Telegram сам отдаёт о профиле. Никаких утечек/баз.
    bio = None
    try:
        bio = (await client.get_chat(user.id)).bio
    except RPCError:
        pass
    photos = None
    try:
        photos = await client.get_chat_photos_count(user.id)
    except RPCError:
        pass
    common = []
    try:
        common = [c.title or str(c.id) for c in await client.get_common_chats(use
    except RPCError:
        pass

    status = getattr(user, "status", None)
    status_txt = STATUS_RU.get(getattr(status, "name", ""), "скрыт")

    profile = [
        Field("Имя", full_name(user), True),
        Field("Username", f"@{user.username}" if user.username else "нет", True),
        Field("ID", str(user.id), True),
        Field("Био", bio or "пусто", True),
    ]
    if getattr(user, "phone_number", None):
        profile.append(Field("Телефон (открыт в профиле)", f"+{user.phone_number.

    flags = [
        Field("Метка SCAM (Telegram)", _yn(getattr(user, "is_scam", False))),
        Field("Метка FAKE (Telegram)", _yn(getattr(user, "is_fake", False))),
        Field("Верифицирован", _yn(getattr(user, "is_verified", False))),
        Field("Premium", _yn(getattr(user, "is_premium", False))),
        Field("Бот", _yn(getattr(user, "is_bot", False))),
        Field("Удалён", _yn(getattr(user, "is_deleted", False))),
    ]
    activity = [
        Field("Последняя активность", status_txt),
        Field("Фото в профиле", str(photos) if photos is not None else "н/д"),
        Field("Общие чаты", str(len(common))),
```

```
    ]
    if common:
        activity.append(Field("Список общих чатов", ", ".join(common[:10]), True)

    verdict = (
        "🔴 У аккаунта есть пометки Telegram (SCAM/FAKE)"
        if getattr(user, "is_scam", False) or getattr(user, "is_fake", False)
        else "🟢 Пометок Telegram не найдено (это не оценка репутации)"
    )
    return Report(
        target_chat_id=chat_id,
        sections=[
            ("👤 Профиль", profile),
            ("🏷️ Метки и статус", flags),
            ("📊 Активность", activity),
            ("📋 Итог", [Field("Вердикт", verdict)]),
        ],
        sources=["Telegram API (публичные данные профиля)"],
    )


def render_plain(report: Report) -> str:
    """Полная версия — для владельца (без Markdown, со всеми данными и источникам
    out = ["🛡️ ANTI-SCAM CHECK | Отчёт по аккаунту", "━━━━━━━━━━━━━━━━━━━━"]
    for title, fields in report.sections:
        out.append(f"\n{title}")
        out += [f"  • {f.label}: {f.value}" for f in fields]
    out.append("\n🔎 Источники: " + "; ".join(report.sources))
    return "\n".join(out)


def render_public(report: Report) -> str:
    """Публичная версия: чувствительные значения в ||спойлере||, источники скрыты
    out = ["🛡️ **ANTI-SCAM CHECK | Отчёт по аккаунту**", "━━━━━━━━━━━━━━━━━━━━"]
    for title, fields in report.sections:
        out.append(f"\n**{title}**")
        for f in fields:
            val = f"||{esc(f.value)}||" if f.sensitive else esc(f.value)
            out.append(f"  • {f.label}: {val}")
    out.append("\n🔒 Источники: скрыты")
    return "\n".join(out)


async def cmd_info(_, message: Message):
    arg = args_of(message)[0] if args_of(message) else None
    user = await resolve_target(message, arg)
    chat_id = message.chat.id
```

```
    if not user:
        await safe_edit(message, f"ℹ️ Использование: `{P}info [ник/id]`, ответом
        _bg(_delete_later(message, 5))
        return
    await drop(message)
    await flash(chat_id, "🔎 Отчёт сформирован и отправлен тебе в ЛС (бот + Избра

    report = await collect_report(user, chat_id)
    rid = uuid.uuid4().hex[:8]
    REPORTS[rid] = report
    plain = render_plain(report)

    # 1) Копия в «Избранное» (юзербот не может ставить инлайн-кнопки)
    try:
        await client.send_message("me", plain + f"\n\n🆔 Report ID: {rid}", parse
    except RPCError as e:
        log.warning("saved messages failed: %s", e)
    # 2) Бот присылает то же самое с кнопкой «📤 Опубликовать»
    if _report_sender:
        try:
            await _report_sender(rid, plain)
        except Exception as e:  # noqa: BLE001
            log.warning("report sender failed: %s", e)
            await flash("me", "⚠️ Бот не смог написать тебе. Открой бота и нажми


async def publish_report(rid: str) -> bool:
    """Вызывается ботом по кнопке «📤 Опубликовать»: шлёт спойлер-версию в целевс
    report = REPORTS.get(rid)
    if not report or client is None:
        return False
    await client.send_message(report.target_chat_id, render_public(report), parse
    return True


def create_client() -> Client:
    """Создаёт юзербота и навешивает хендлеры. Вызывать уже внутри async-кода."""
    global client
    client = Client("userbot", api_id=API_ID, api_hash=API_HASH, session_string=S

    def cmd(names):
        return filters.me & filters.command(names, prefixes=P)  # только твои соо

    table = [
        (["spam"], cmd_spam), (["heart"], cmd_heart), (["stop"], cmd_stop),
        (["slots"], cmd_slots), (["dice", "darts"], cmd_dice),
        (["freeze", "snos"], cmd_freeze), (["unfreeze"], cmd_unfreeze),
```

```
        (["mute"], cmd_mute), (["unmute"], cmd_unmute),
        (["info"], cmd_info),
    ]
    for names, func in table:
        client.add_handler(MessageHandler(func, cmd(names)), group=0)
    # Автоудаление замьюченных — отдельная группа, чтобы не конфликтовать с коман
    client.add_handler(MessageHandler(delete_muted, muted_filter), group=1)
    return client


# ═══════════════════════════════════════════════════════════════════════
#                                  БОТ (aiogram)
# ═══════════════════════════════════════════════════════════════════════
router = Router()

START_TEXT = f"""👋 <b>Привет!</b> Это <b>{BOT_NAME}</b> — панель управления твои

Юзербот работает от лица твоего аккаунта: пишешь команду в любом чате — он выполн

<b>💬 Спам и романтика</b>
<code>{P}spam [1-{SPAM_MAX_COUNT}] [текст]</code> — серия сообщений (пауза 1.5–2.
<code>{P}heart [текст]</code> — сердце из смайликов с текстом внутри
<code>{P}stop</code> — мгновенно останавливает всё

<b>🎮 Игры</b>
<code>{P}slots</code> — казино с очками
<code>{P}dice</code> / <code>{P}darts</code> — кубик / дротик Telegram

<b>🥶 Заморозка и снос (розыгрыш)</b>
<code>{P}freeze</code> — анимация: аккаунт будет заморожен в течение {FREEZE_MINU
<code>{P}snos</code> — анимация: аккаунт будет снесён в течение {SNOS_HOURS_RANGE
<code>{P}unfreeze [Task ID]</code> — отмена (или твоя кнопка «🛑 Остановить» под

<b>🔨 Мьют</b>
<code>{P}mute 10 min</code> — автоудаление сообщений собеседника (в личке) или ав
<code>{P}unmute</code> — снять мьют

<b>🔎 Инфо</b>
<code>{P}info</code> — отчёт по публичным данным профиля приходит сюда с кнопкой

<i>Цель: ответ (reply) на сообщение, в личке — собеседник, либо ник/ID в команде.

HOWTO_TEXT = f"""📖 <b>Как подключить юзербота к чату</b>

Юзербот — это твой аккаунт, поэтому «добавлять» его в чаты не нужно: он работает

<b>1. Ключи API</b>
```

```
my.telegram.org → API development tools → создай приложение → скопируй <b>API_ID<

<b>2. Строка сессии</b>
Запусти <code>python main.py session</code> (Termux / ПК), введи API_ID, API_HASH

<b>3. Настройка бота в @BotFather</b> (нужна для {P}freeze и {P}snos)
/setinline → выбери бота → любой текст-подсказка
/setinlinefeedback → выбери бота → <b>Enabled</b>

<b>4. Переменные окружения (Render → Environment)</b>
<code>BOT_TOKEN</code>, <code>API_ID</code>, <code>API_HASH</code>, <code>SESSION

<b>5. Запуск</b>
Команда: <code>python main.py</code>. Напиши боту /start, иначе он не сможет прис

<b>6. Проверка</b>
В «Избранном» отправь <code>{P}heart привет</code>.

<b>Важно</b>
• Мьют в группах требует прав админа на удаление сообщений.
• Анимация freeze/snos отправляется через бота — в сообщении будет пометка «via @
• За спам Telegram может ограничить аккаунт."""


def kb_start() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="📖 К


def kb_back() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="⬅️ Н


@router.message(CommandStart())
async def on_start(message: BotMessage):
    await message.answer(START_TEXT, reply_markup=kb_start())


@router.callback_query(F.data == "howto")
async def on_howto(call: CallbackQuery):
    await call.message.edit_text(HOWTO_TEXT, reply_markup=kb_back())
    await call.answer()


@router.callback_query(F.data == "start")
async def on_back(call: CallbackQuery):
    await call.message.edit_text(START_TEXT, reply_markup=kb_start())
    await call.answer()
```

```
# ──────────────────────────── .info: публикация отчёта ────────────────────────────
@router.callback_query(F.data.startswith("publish:"))
async def on_publish(call: CallbackQuery):
    """Кнопка «📤 Опубликовать» — только для владельца."""
    if call.from_user.id != OWNER_ID:
        await call.answer("Недоступно", show_alert=True)
        return
    rid = call.data.split(":", 1)[1]
    try:
        ok = await publish_report(rid)
    except Exception as e:  # noqa: BLE001
        await call.answer(f"Ошибка: {e}", show_alert=True)
        return
    if not ok:
        await call.answer("Отчёт не найден (возможно, был перезапуск)", show_aler
        return
    await call.message.edit_reply_markup(
        reply_markup=InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(
    )
    await call.answer("Опубликовано (данные под спойлерами)")


@router.callback_query(F.data == "noop")
async def on_noop(call: CallbackQuery):
    await call.answer()


def make_report_sender(bot: Bot) -> ReportSender:
    async def sender(report_id: str, plain_text: str) -> None:
        kb = InlineKeyboardMarkup(
            inline_keyboard=[[InlineKeyboardButton(text="📤 Опубликовать", callba
        )
        await bot.send_message(OWNER_ID, plain_text, parse_mode=None, reply_marku

    return sender


# ──────────────────────────── FREEZE / SNOS: анимация ────────────────────────────
MODES = {
    "freeze": {
        "title": "🧊 FREEZE PROTOCOL",
        "confirm": "⚠️ <b>Подтвердите запуск алгоритма блокировки аккаунта</b>",
        "stages": ["Инициализация", "Проверка параметров", "Применение ограничени
        "final": "🧊 <b>{target} будет заморожен в течение {label}</b>",
    },
```

```
    "snos": {
        "title": "🗑️ ACCOUNT TERMINATION",
        "confirm": "⚠️ <b>Подтвердите запуск процедуры удаления аккаунта</b>",
        "stages": ["Сбор данных аккаунта", "Формирование запроса", "Передача в об
        "final": "🗑️ <b>{target} будет снесён в течение {label}</b>",
    },
}


def target_label(info: dict) -> str:
    """@ник цели (или имя, если ника нет) — уже экранировано для HTML."""
    u = info["user"]
    return escape(f"@{u.username}" if u.username else full_name(u))


def kb_stop(task_id: str) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🛑 C


def plate(info: dict, task_id: str) -> str:
    user = info["user"]
    username = f"@{user.username}" if user.username else "—"
    return (
        f"<b>{MODES[info['mode']]['title']}</b>\n"
        "━━━━━━━━━━━━━━\n"
        f"👤 <b>Цель:</b> {escape(full_name(user))}\n"
        f"🔗 <b>Username:</b> {escape(username)}\n"
        f"🆔 <b>ID:</b> <code>{user.id}</code>\n"
        f"🗂️ <b>Task ID:</b> <code>{task_id}</code>\n"
        "━━━━━━━━━━━━━━\n"
    )


def progress_bar(pct: int, width: int = 12) -> str:
    filled = round(width * pct / 100)
    return "█" * filled + "░" * (width - filled) + f" {pct}%"


@router.inline_query()
async def on_inline(q: InlineQuery):
    """Юзербот делает запрос `fz <TaskID>` — отвечаем одним результатом с кнопкой
    parts = (q.query or "").split()
    info = FREEZES.get(parts[1]) if len(parts) == 2 and parts[0] == "fz" else Non
    if q.from_user.id != OWNER_ID or not info:
        await q.answer([], cache_time=0, is_personal=True)
        return
    tid = parts[1]
```

```
    article = InlineQueryResultArticle(
        id=tid,
        title="Запуск",
        input_message_content=InputTextMessageContent(message_text=plate(info, ti
        reply_markup=kb_stop(tid),  # кнопка нужна, чтобы Telegram прислал inline
    )
    await q.answer([article], cache_time=0, is_personal=True)


@router.chosen_inline_result()
async def on_chosen(res: ChosenInlineResult, bot: Bot):
    """Сообщение ушло в чат — получаем inline_message_id и запускаем анимацию."""
    info = FREEZES.get(res.result_id)
    if not info or info["cancelled"] or not res.inline_message_id:
        return
    info["inline_id"] = res.inline_message_id
    info["task"] = asyncio.create_task(freeze_job(bot, res.result_id))


async def freeze_job(bot: Bot, tid: str):
    """Обёртка: ошибки анимации попадают в лог, а не пропадают молча."""
    try:
        await _freeze_run(bot, tid)
    except asyncio.CancelledError:
        raise
    except Exception:  # noqa: BLE001
        log.exception("freeze animation failed")


async def _freeze_run(bot: Bot, tid: str):
    info = FREEZES[tid]
    mode = MODES[info["mode"]]
    head = plate(info, tid)

    async def show(text: str):
        try:
            await bot.edit_message_text(
                text, inline_message_id=info["inline_id"], parse_mode=ParseMode.H
            )
        except TelegramRetryAfter as e:
            await asyncio.sleep(e.retry_after)
        except TelegramBadRequest:
            pass  # «сообщение не изменилось» и т.п.

    async def pause():
        await asyncio.sleep(random.uniform(*ANIM_DELAY_RANGE))
```

```
    await show(head + "🔍 Поиск аккаунта в базе...")
    await pause()
    await show(head + mode["confirm"])
    await pause()
    await show(head + "✅ Подтверждено. Запуск...")
    await pause()

    for pct in range(0, 101, 10):
        stage = mode["stages"][min(pct // 26, len(mode["stages"]) - 1)]
        await show(head + f"⚙️ {stage}\n<code>{progress_bar(pct)}</code>")
        await pause()

    left = info["total_seconds"]
    for _ in range(4):  # «тикающий» финальный таймер
        h, rem = divmod(left, 3600)
        m, sec = divmod(rem, 60)
        await show(
            head
            + mode["final"].format(target=target_label(info), label=info["label"]
            + f"\n⏳ Осталось: <code>{h:02d}:{m:02d}:{sec:02d}</code>"
        )
        left -= random.randint(1, 2)
        await pause()


@router.callback_query(F.data.startswith("fz_stop:"))
async def on_fz_stop(call: CallbackQuery, bot: Bot):
    """Кнопка «Остановить»: у владельца реально отменяет, у остальных — только то
    tid = call.data.split(":", 1)[1]
    if call.from_user.id == OWNER_ID:
        await cancel_freezes(bot, tid)
        await call.answer("✅ Остановлено")
    else:
        await call.answer("⏳ Запрос на остановку принят. Обработка...")


async def cancel_freezes(bot: Bot, tid: Optional[str] = None) -> int:
    """Отмена одной (tid) или всех активных анимаций. Возвращает число отменённых
    if tid:
        ids = [tid]
    else:
        ids = [t for t, i in FREEZES.items() if i["task"] and not i["task"].done(
    n = 0
    for t in ids:
        info = FREEZES.get(t)
        if not info or info["cancelled"]:
            continue
```

```
            info["cancelled"] = True
            task = info["task"]
            if task and not task.done():
                task.cancel()
                await asyncio.gather(task, return_exceptions=True)
            if info["inline_id"]:
                try:
                    await bot.edit_message_text(
                        plate(info, t) + "🚫 <b>Статус:</b> Отменено владельцем",
                        inline_message_id=info["inline_id"], parse_mode=ParseMode.HTM
                    )
                except (TelegramBadRequest, TelegramRetryAfter):
                    pass
            n += 1
        return n


# ═══════════════════════════════════════════════════════════════════════
#                          ЗАПУСК (+ мини-сервер для Render)
# ═══════════════════════════════════════════════════════════════════════
async def start_health_server() -> Optional[web.AppRunner]:
    """Render Web Service требует открытый порт. Если переменной PORT нет — не за
    port = os.getenv("PORT")
    if not port:
        return None

    async def ok(_request):
        return web.Response(text="ok")

    app = web.Application()
    app.router.add_get("/", ok)
    app.router.add_get("/health", ok)
    runner = web.AppRunner(app)
    await runner.setup()
    await web.TCPSite(runner, "0.0.0.0", int(port)).start()
    log.info("Health-сервер на порту %s", port)
    return runner


def check_env() -> None:
    required = {"BOT_TOKEN": BOT_TOKEN, "API_ID": API_ID, "API_HASH": API_HASH, "
    missing = [k for k, v in required.items() if not v]
    if missing:
        raise SystemExit(f"❌ Не заданы переменные окружения: {', '.join(missing)


async def main() -> None:
```

```
    global OWNER_ID, BOT_USERNAME, _report_sender, _freeze_canceller
    check_env()
    runner = await start_health_server()

    bot = Bot(BOT_TOKEN, default=DefaultBotProperties(parse_mode=ParseMode.HTML))
    dp = Dispatcher()
    dp.include_router(router)

    await bot.delete_webhook(drop_pending_updates=True)
    bot_me = await bot.get_me()

    ub = create_client()
    await ub.start()
    me = await ub.get_me()
    OWNER_ID = me.id  # владелец = аккаунт юзербота (отдельная переменная не нужн

    BOT_USERNAME = bot_me.username
    _report_sender = make_report_sender(bot)
    _freeze_canceller = lambda tid: cancel_freezes(bot, tid)  # noqa: E731

    log.info("Юзербот: %s (id %s) | Бот: @%s", me.first_name, me.id, bot_me.usern
    try:  # приветствие владельцу (сработает, если ты уже нажимал /start у бота)
        await bot.send_message(OWNER_ID, "✅ Юзербот и бот запущены. Список коман
    except Exception:  # noqa: BLE001
        log.info("Владелец ещё не нажал /start у бота — приветствие пропущено")

    try:
        await dp.start_polling(bot)
    finally:
        await ub.stop()
        await bot.session.close()
        if runner:
            await runner.cleanup()


async def generate_session() -> None:
    """python main.py session — выдаёт SESSION_STRING (интерактивно: номер, код,
    api_id = int(input("API_ID: ").strip())
    api_hash = input("API_HASH: ").strip()
    async with Client("gen", api_id=api_id, api_hash=api_hash, in_memory=True) as
        print("\nТвоя SESSION_STRING (никому не показывай!):\n")
        print(await app.export_session_string())


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "session":
        asyncio.run(generate_session())
```

```
else:
    asyncio.run(main())
```